# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Artificial Intelligence and Data Science

## ASSESSMENT TOOL 3 – CODING CHALLENGE

| | | | |
|---|---|---|---|
| **Course Code:** | DSA03 | **Course Title:** | Natural Language Processing |
| **CO Assessed:** | CO1 – Imitation (L1) | **Assessment:** | Assessment Tool 3 – Coding Challenge |
| **Weightage:** | 30% of CIA | | |
| **CO5 Statement:** | Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3) | | |
| **Student Name:** | A.ARCHISHMAN | **Reg. No.:** | 192425341 |
| **Section / Batch:** | AI&ML | **Date:** | 27-10-2026 |

**Code of Conduct**

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: ____Archishman____

**Instructions**

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

| Section / Topic | Focus | Q Nos |
|---|---|---|
| **Regular Expression Pattern Matching** | Regular Expressions, Basic Regular Expression Patterns | 1 |
| **Finite State Automata Implementation** | Finite State Automata, Models and Algorithms | 2 |
| **Advanced Regular Expression Search** | Regular Expressions, Basic Regular Expression Patterns | 3 |

## Annexure A – Coding Challenge

### Section 1: Intelligent Email and Password Validator using Regular Expressions

1. **Problem Statement:**

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

**Develop a Python application that validates:**
Email Address
Password
Mobile Number

using Regular Expressions.

**Validation Rules**
Email
Must start with an alphabet.
May contain letters, digits, '.', '_' before '@'.
Domain should contain only alphabets.
Valid extensions: .com, .org, .edu, .net, .in
**Password**
Minimum 8 characters
At least one uppercase letter
At least one lowercase letter
At least one digit
At least one special character (@,#,$,%,&,!)
**Mobile Number**
Must contain exactly 10 digits.
First digit should be between 6–9.
**Expected Output**

Display:

Valid Email / Invalid Email
Strong Password / Weak Password
Valid Mobile Number / Invalid Mobile Number

### Section 2: Design a Finite State Automata (FSA) Simulator

2. **Problem Statement**

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

**The program should:**

- Accept the DFA description
    - States
    - Input alphabet
    - Transition table
    - Initial state
    - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

**Example**

Language:
Strings ending with "ab"

**Input**
abaab

**Output**
Transition Path:
q0 → q1 → q0 → q1 → q2
Accepted

## Section 3: Smart Pattern Matching Engine using Regular Expressions

### 3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

**The system should support:**
Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

**Example Input**
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

**User Menu**
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix

**Expected Output**
Display all matching patterns based on user selection.

## Rubrics for Calculation

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| **Problem Understanding** | 20% | Clearly understands problem constraints, edge cases, and competitive requirements | Understands problem with minor gaps in constraints | Partial understanding; misses key constraints | Misinterprets problem or constraints |
| **Algorithm** | 25% | Selects optimal algorithm with best time and space complexity for competitive constraints | Uses efficient algorithm with minor scope for improvement | Uses basic/inefficient approach | No clear algorithmic approach |
| **Code Implementation** | 20% | Writes correct, efficient, and clean code that passes all constraints | Code works with minor errors or inefficiencies | Partially correct code; fails for some cases | Code does not execute or gives incorrect results |
| **Competitive Performance** | 20% | Solves problem within time limits and handles large inputs effectively | Solves within acceptable time with minor delays | Struggles with time limits or larger inputs | Unable to solve within time constraints |
| **Test Case Handling** | 15% | Handles all test cases including edge and hidden cases | Handles most test cases | Misses important edge cases | Fails on multiple test cases |

| Q. No. / Criteria | Problem Understanding | Algorithm | Code Implementation | Competitive Performance | Test Case Handling | Sub. Total | Total % |
|---|---|---|---|---|---|---|---|
| 1 | | | | | | | |
| 2 | | | | | | | |
| 3 | | | | | | | |

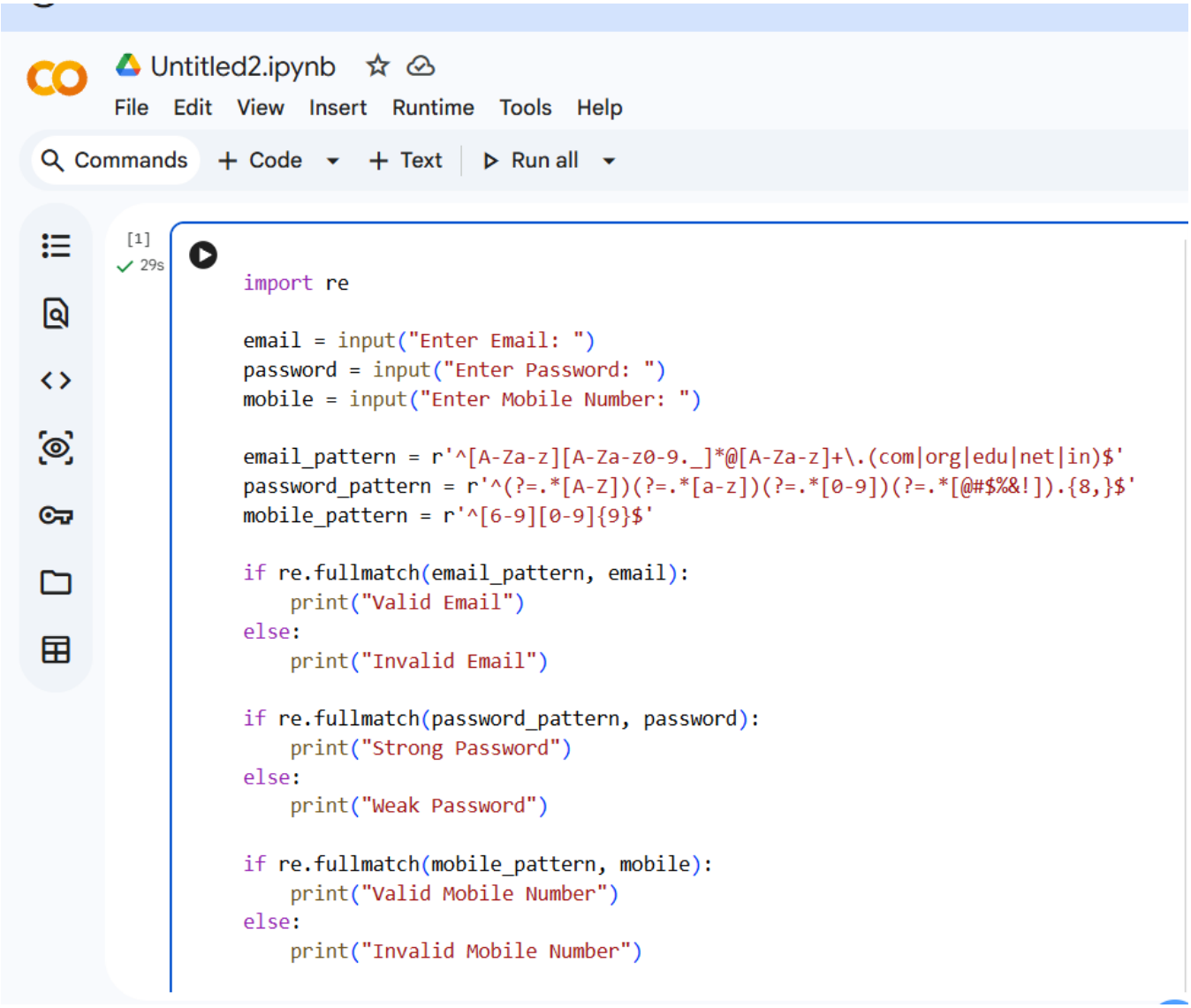
| Faculty In-charge | Course Coordinator | Program Director |
|---|---|---|
| | | |
| Signature: _____ | Signature: _____ | Signature: _____ |
| Date: _____ | Date: _____ | Date: _____ |

Q1. Intelligent Email and Password Validator Using Regular Expressions

## Aim:

To develop a Python program using regular expressions to validate email addresses, passwords, and mobile numbers based on specified rules.

## Program Code



## Output

```
Enter Email: sunnyarchish@gmail.com
Enter Password: Sunny13
Enter Mobile Number: 9987654321
Valid Email
Weak Password
Valid Mobile Number
```

## Result:

Thus, the program successfully validates email addresses, checks password strength, and verifies mobile numbers using regular expressions.

Q2. Design a Finite State Automata (FSA) Simulator

## Aim:

To design and implement a Deterministic Finite Automaton (DFA) simulator in Python to process input strings and determine whether they are accepted or rejected.

## Program Code

[2]
24s

```
def simulate_dfa(string):
    transitions = {
        'q0': {'a': 'q1', 'b': 'q0'},
        'q1': {'a': 'q1', 'b': 'q2'},
        'q2': {'a': 'q1', 'b': 'q0'}
    }

    current_state = 'q0'
    path = [current_state]

    for symbol in string:
        if symbol not in ['a', 'b']:
            print("Invalid input! Use only a and b.")
            return

        current_state = transitions[current_state][sy
        path.append(current_state)

    print("Input String:", string)
    print("Transition Path:", " -> ".join(path))

    if current_state == 'q2':
        print("Accepted")
    else:
        print("Rejected")
```

## Output

```
Enter number of strings: 2
Enter string: abaab
Input String: abaab
Transition Path: q0 -> q1 -> q2 -> q1 -> q1 -> q2
Accepted

Enter string: abaa
Input String: abaa
Transition Path: q0 -> q1 -> q2 -> q1 -> q1
Rejected
```

## Result:

Thus, the DFA simulator successfully processes input strings, displays the state transition path, and accepts strings ending with ab while rejecting other strings.

Q3. Smart Pattern Matching Engine Using Regular Expressions

## Aim:

To develop a Python-based text search engine using regular expressions to identify dates, phone numbers, hashtags, mentions, prefixes, suffixes, and words.

## Program Code

```
import re

text = """
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing
"""

while True:
    print("\n1. Search Date")
    print("2. Search Phone Number")
    print("3. Search Hashtag")
    print("4. Search Mention")
    print("5. Search Prefix")
    print("6. Search Suffix")
    print("7. Search Word")
    print("8. Exit")

    choice = input("Enter your choice: ")

    if choice == '1':
        pattern = r'\b\d{2}/\d{2}/\d{4}\b'

    elif choice == '2':
        pattern = r'\b[6-9]\d{9}\b'

    elif choice == '3':
        pattern = r'#[A-Za-z0-9_]+'

    elif choice == '4':
        pattern = r'@[A-Za-z0-9_]+'

    elif choice == '5':
        prefix = input("Enter prefix: ")
        pattern = r'\b' + re.escape(prefix) + r'\w*'

    elif choice == '6':
        suffix = input("Enter suffix: ")
        pattern = r'\b\w*' + re.escape(suffix) + r'\b'

    elif choice == '7':
        word = input("Enter word: ")
        pattern = r'\b' + re.escape(word) + r'\b'
```

## Output

```
====

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter your choice: 1
Matches Found: ['12/09/2026']

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter your choice: 4
Matches Found: ['@OpenAI']

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter your choice: 7
Enter word: SUNNY
No matches found.

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter your choice:
```

## Result:

Thus, the program successfully searches the given text for the selected patterns and displays all matching results using regular expressions.